

Домашка 04 — Мережі. Частина 1

Автор: Андрій Пономарьов / Claude Code **Завдання:** пройти шлях `dig` → `route` → `traceroute` для довільного сайту, зняти `tcpdump` під час `curl` і знайти TCP handshake (SYN, SYN-ACK, ACK), записати у файл кількість хопів, шлюз і його MAC, пояснити, чому MAC змінюється, а IP — ні.

Обраний сайт: **github.com**

Примітка: робота виконувалась на macOS, де немає команди `ip route get` (вона з linux-пакета `iproute2`). Використано нативний еквівалент — `route -n get`.

1. dig — DNS-резолв

```
$ dig github.com

;; ANSWER SECTION:
github.com.      60 IN  A   140.82.121.4

;; Query time: 50 msec
;; SERVER: 192.168.50.2#53(192.168.50.2)
```

Що це означає: DNS-сервер 192.168.50.2 (локальний, у мережі роутера) відповів, що `github.com` зараз резолвиться в IP 140.82.121.4. TTL = 60 секунд — запис короткоживучий, GitHub ротує адреси між 140.82.121.3–.5 для балансування навантаження (в `traceroute` нижче DNS вже видав .3). Повний вивід: [docs/dig-output.txt](#).

2. route — куди піде пакет

```
$ route -n get github.com
route to: 140.82.121.4
gateway: 10.20.50.1
interface: en0
flags: <UP,GATEWAY,DONE,STATIC,PRCLONING,GLOBAL>
mtu: 1500
```

Що це означає: адреса 140.82.121.4 не в моїй локальній мережі, тому пакет піде через шлюз за замовчуванням 10.20.50.1 з інтерфейсу `en0` (Wi-Fi), MTU 1500 байтів. Повний вивід разом з `netstat -rn ta arp -a`: [docs/route-output.txt](#).

3. traceroute — шлях пакета

```
$ traceroute -I -n -w 2 -m 20 github.com
traceroute to github.com (140.82.121.3), 20 hops max, 48 byte packets
 1  10.20.50.1  2.872 ms  4.571 ms  4.676 ms
 2  10.12.18.1  4.468 ms  4.667 ms  4.415 ms
 3  10.1.1.3    4.604 ms  5.286 ms  5.603 ms
 4  91.226.254.3 5.074 ms  6.119 ms  3.371 ms
```

```

5 185.151.86.16 4.652 ms 9.569 ms 6.392 ms
6 185.1.226.180 56.857 ms 56.701 ms 55.604 ms
7 80.81.196.80 44.399 ms 45.139 ms 46.686 ms
8 * * *
9 * * *
10 140.82.121.3 46.064 ms 48.998 ms 46.365 ms

```

Що це означає: до GitHub — **10 хопів**. Перший хоп — мій шлюз 10.20.50.1, хопи 2–3 — приватні адреси мережі провайдера, з хопу 4 починаються публічні адреси. Стрибок затримки з ~5 мс до ~55 мс на хопі 6 (185.1.226.180 — це peering-точка DTEL-IX) — момент, коли трафік виходить за кордон. Хопи 8–9 (* * *) — маршрутизатори GitHub, які не відповідають на ICMP TTL exceeded. Звичайний traceroute -n (UDP-проби) показує лише перші 7 хопів — GitHub відкидає UDP на високі порти, тому фінал шляху видно тільки по ICMP (-I). Повний вивід обох варіантів: [docs/traceroute-output.txt](#).

4. tcpdump під час curl — TCP handshake

Захоплення на en0 з фільтром на мережу GitHub (140.82.112.0/20), паралельно виконувався curl -s -o /dev/null https://github.com:

```

$ sudo tcpdump -i en0 -nn "tcp and net 140.82.112.0/20"

19:32:46.921664 IP 10.20.50.89.52079 > 140.82.121.3.443: Flags [SEW], seq 1977315521, win 6553
5, options [mss 1460,...], length 0
19:32:46.967969 IP 140.82.121.3.443 > 10.20.50.89.52079: Flags [S.E], seq 67592, ack 197731552
2, win 65535, options [mss 1436,...], length 0
19:32:46.968157 IP 10.20.50.89.52079 > 140.82.121.3.443: Flags [.], ack 1, win 2070, options
[...], length 0

```

Що це означає: це і є three-way handshake:

1. **SYN** — Flags [SEW]: мій комп'ютер (10.20.50.89, порт 52079) шле SYN на 140.82.121.3:443 з початковим seq 1977315521. Літера s — SYN, E і W (ECE+CWR) — узгодження ECN, розширення для контролю перевантаження.
2. **SYN-ACK** — Flags [S.E]: сервер відповідає своїм SYN (seq 67592) і підтверджує мій — ack 1977315522 (мій seq + 1). Крапка у прапорах tcpdump означає ACK.
3. **ACK** — Flags [.]: я підтверджую SYN сервера, з'єднання встановлене. Handshake зайняв ~46 мс — рівно один RTT до GitHub (збігається з traceroute).

Одразу після цього йде пакет Flags [P.], length 320 — це вже TLS ClientHello поверх встановленого з'єднання. Ключові фрагменти дампа: [docs/tcpdump-output.txt](#).

5. Файл з підсумком

Кількість хопів, шлюз і його MAC записані у [docs/network-info.txt](#):

- **Хопів до github.com:** 10
- **Шлюз:** 10.20.50.1 (en0)
- **MAC шлюзу:** b0:e4:d5:43:f3:c8

6. Чому MAC змінюється, а IP — ні

IP-адреси в заголовку пакета — це адреси кінцевих точок з'єднання (мій комп'ютер і сервер GitHub), вони працюють на мережевому рівні (L3) і задають маршрут «звідки й куди» на всьому шляху. MAC-адреси працюють на каналному рівні (L2) і мають сенс лише в межах одного локального сегмента мережі — наприклад, між моїм ноутбуком і Wi-Fi-роутером. Кожен маршрутизатор на шляху розпаковує Ethernet-кадр, дивиться на IP-адресу призначення і запаковує пакет у новий кадр: source MAC стає адресою самого маршрутизатора, а destination MAC — адресою наступного хопу (яку він дізнається через ARP). Тому на кожному з 10 хопів пара MAC-адрес інша, а пара IP-адрес у заголовку пакета лишається незмінною від початку до кінця. Єдиний виняток — NAT на домашньому роутері, який підмінює мій приватний source IP (10.20.50.x) на публічну адресу, але destination IP сервера все одно не змінюється ніде.

Опціональна частина

7. nc -zv: відкритий / закритий / фільтрований порт

```
$ nc -zv github.com 443
Connection to github.com port 443 [tcp/https] succeeded!

$ nc -zv 127.0.0.1 65000
nc: connectx to 127.0.0.1 port 65000 (tcp) failed: Connection refused

$ time nc -zv -G 5 github.com 8080
nc: connectx to github.com port 8080 (tcp) failed: Operation timed out
(5.013 total)
```

Порівняння: три стани порту дають три різні реакції:

Порт	Результат	Час	Що сталося на рівні TCP
github.com:443 (відкритий)	succeeded	миттєво	SYN → SYN-ACK
127.0.0.1:65000 (закритий)	Connection refused	миттєво	SYN → RST
github.com:8080 (фільтрований)	Operation timed out	5 с (таймаут)	SYN → тиша

Закритий порт «чесний» — ОС одразу відповідає RST. Фільтрований — firewall у режимі DROP мовчки з'їдає SYN, і клієнт здається лише по таймауту. Саме за цією різницею nmap відрізняє closed від filtered. Повний вивід: [docs/nc-output.txt](https://github.com/nmap/nmap/blob/master/README.md#nc-output).

8. MTU і ping -s

MTU en0 = 1500 (видно у `route -n get`). Розмір ICMP-пакета = payload + 8 (ICMP) + 20 (IP), тож межа payload без фрагментації: 1500 – 28 = **1472**. Прапор `-D` ставить DF-біт (Don't Fragment):

```
$ ping -c 3 -D -s 1472 github.com      # 1472+28 = 1500 — рівно MTU
1480 bytes from 140.82.121.4: ... time=46.133 ms  → 0% loss

$ ping -c 3 -D -s 1473 github.com      # 1501 > MTU, DF заборонив різати
ping: sendto: Message too long          → 100% loss

$ ping -c 3 -s 2000 github.com          # 2028 > MTU, але БЕЗ DF
2008 bytes from 140.82.121.4: ... time=45.309 ms  → 0% loss
```

Що це означає: 1472 байти проходять рівно в MTU; +1 байт з DF-бітом — ядро навіть не відправляє пакет («Message too long»); той самий великий пакет без DF проходить, бо IP-рівень прозоро розрізає його на два фрагменти, а приймальна сторона склеює (RTT не змінився). На цьому механізмі побудовано Path MTU Discovery: TCP завжди ставить DF і зменшує сегмент, коли з маршруту прилітає ICMP “fragmentation needed”. Повний вивід: [docs/mtu-ping-output.txt](#).

9. Wireshark — той самий handshake у GUI

Встановлено Wireshark 4.6.7 (`brew install --cask wireshark`; разом з ним ставиться ChmodBPF, після чого захоплення працює без `sudo`). Запуск одразу із захопленням і тим самим фільтром, що в `tcpdump`:

```
$ open -a Wireshark --args -i en0 -k -f "tcp and net 140.82.112.0/20"
$ curl -s -o /dev/null https://github.com
```

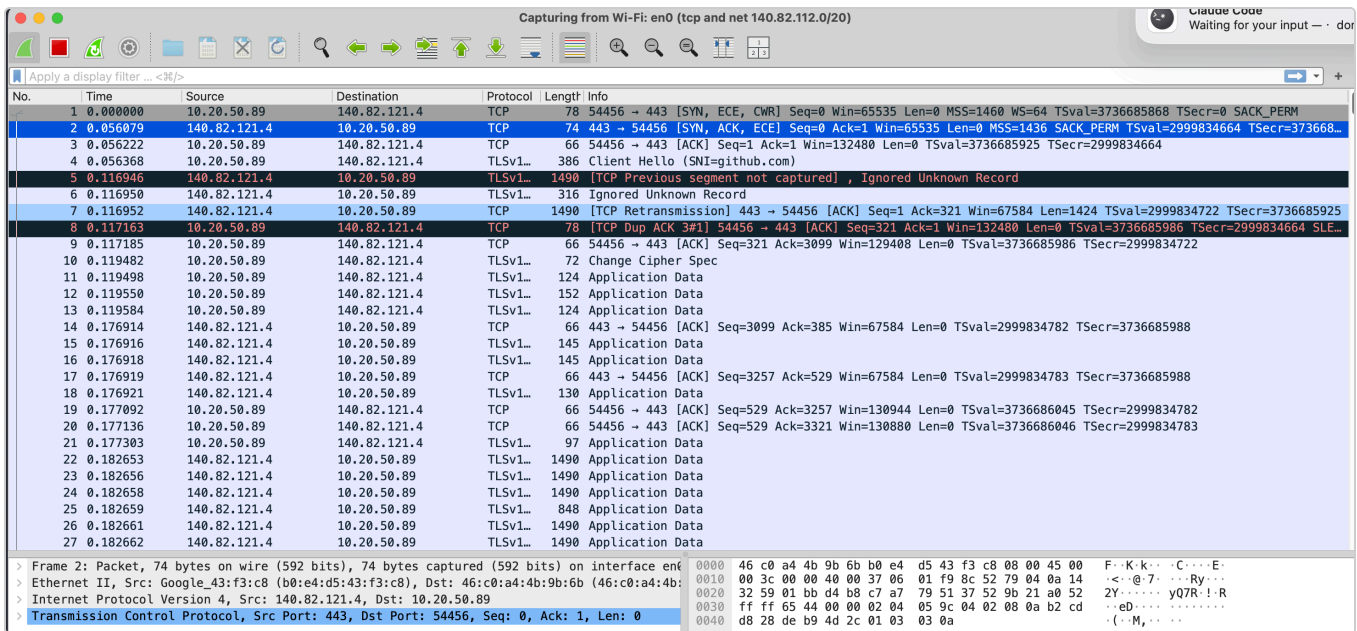
Перші три пакети списку — той самий three-way handshake, четвертий — TLS Client Hello з `SNI=github.com`:

Handshake у Wireshark

Що видно: Wireshark розписує прапори повними назвами — те, що `tcpdump` показував як `[SEW]` / `[S.E]` / `[.]`, тут підписано `[SYN, ECE, CWR]` / `[SYN, ACK, ECE]` / `[ACK]`. У панелі деталей для Frame 1 (SYN) видно Ethernet-заголовок: Src 46:c0:a4:4b:9b:6b (MAC ноутбука) → Dst b0:e4:d5:43:f3:c8 —

це MAC шлюзу з розділу 5, жива ілюстрація до пояснення «MAC vs IP»: кадр до GitHub фізично адресований роутеру. Wireshark навіть підказує вендора шлюзу — Google_43:f3:c8 (Nest Wifi).

Дзеркальна картина у Frame 2 (SYN-ACK): там source MAC — вже шлюз b0:e4:d5:43:f3:c8, а destination — ноутбук:

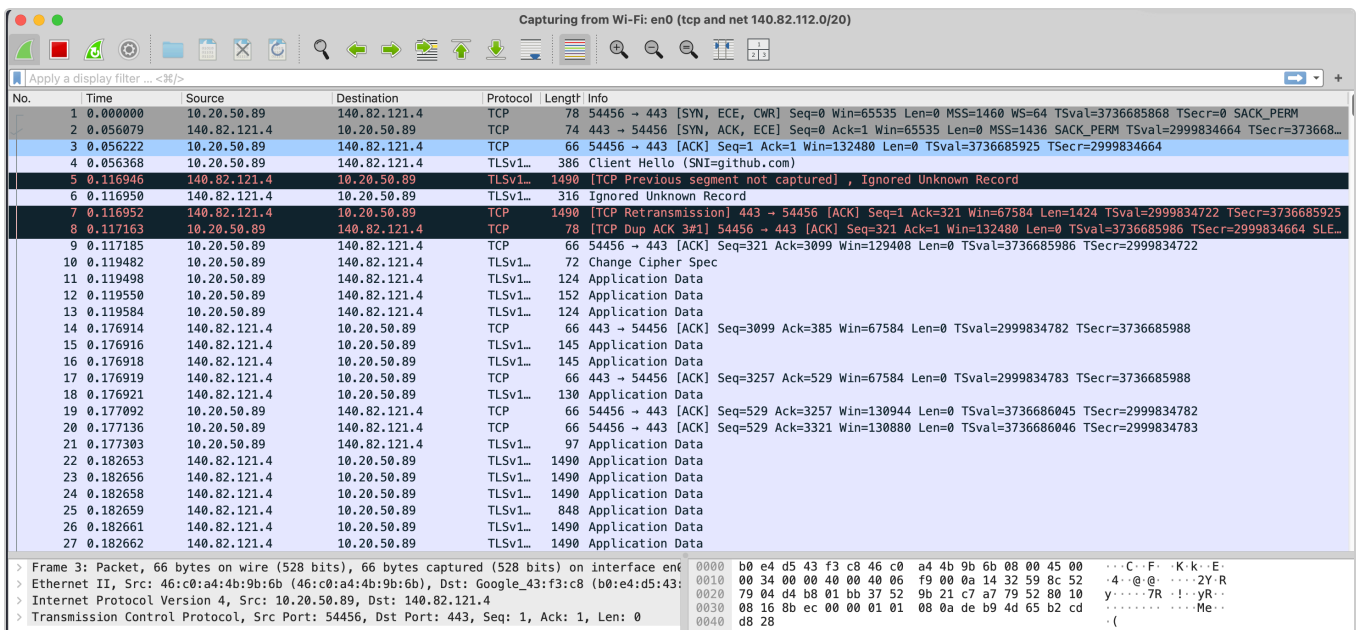


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.20.50.89	140.82.121.4	TCP	78	54456 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3736685868 TSecr=0 SACK_PERM
2	0.056079	140.82.121.4	10.20.50.89	TCP	74	443 → 54456 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1436 SACK_PERM TSval=2999834664 TSecr=373668...
3	0.056222	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=1 Ack=1 Win=132480 Len=0 TSval=3736685925 TSecr=2999834664
4	0.056368	10.20.50.89	140.82.121.4	TLSPV1...	386	Client Hello (SNI=github.com)
5	0.116946	140.82.121.4	10.20.50.89	TLSPV1...	1490	[TCP Previous segment not captured] , Ignored Unknown Record
6	0.116950	140.82.121.4	10.20.50.89	TLSPV1...	316	Ignored Unknown Record
7	0.116952	140.82.121.4	10.20.50.89	TCP	1490	[TCP Retransmission] 443 → 54456 [ACK] Seq=1 Ack=321 Win=67584 Len=1424 TSval=2999834722 TSecr=3736685925
8	0.117163	10.20.50.89	140.82.121.4	TCP	78	[TCP Dup ACK 3#1] 54456 → 443 [ACK] Seq=321 Ack=1 Win=132480 Len=0 TSval=3736685986 TSecr=2999834664 SLE...
9	0.117185	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=321 Ack=3099 Win=129408 Len=0 TSval=3736685986 TSecr=2999834722
10	0.119482	10.20.50.89	140.82.121.4	TLSPV1...	72	Change Cipher Spec
11	0.119498	10.20.50.89	140.82.121.4	TLSPV1...	124	Application Data
12	0.119550	10.20.50.89	140.82.121.4	TLSPV1...	152	Application Data
13	0.119584	10.20.50.89	140.82.121.4	TLSPV1...	124	Application Data
14	0.176914	140.82.121.4	10.20.50.89	TCP	66	443 → 54456 [ACK] Seq=3099 Ack=385 Win=67584 Len=0 TSval=2999834782 TSecr=3736685988
15	0.176916	140.82.121.4	10.20.50.89	TLSPV1...	145	Application Data
16	0.176918	140.82.121.4	10.20.50.89	TLSPV1...	145	Application Data
17	0.176919	140.82.121.4	10.20.50.89	TCP	66	443 → 54456 [ACK] Seq=3257 Ack=529 Win=67584 Len=0 TSval=2999834783 TSecr=3736685988
18	0.176921	140.82.121.4	10.20.50.89	TLSPV1...	130	Application Data
19	0.177092	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=529 Ack=3257 Win=130944 Len=0 TSval=3736686045 TSecr=2999834782
20	0.177136	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=529 Ack=3321 Win=130880 Len=0 TSval=3736686046 TSecr=2999834783
21	0.177383	10.20.50.89	140.82.121.4	TLSPV1...	97	Application Data
22	0.182653	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
23	0.182656	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
24	0.182658	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
25	0.182659	140.82.121.4	10.20.50.89	TLSPV1...	848	Application Data
26	0.182661	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
27	0.182662	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data

> Frame 2: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface en0
> Ethernet II, Src: Google_43:f3:c8 (b0:e4:d5:43:f3:c8), Dst: 46:c0:a4:4b:9b:6b (46:c0:a4:4b:9b:6b)
> Internet Protocol Version 4, Src: 140.82.121.4, Dst: 10.20.50.89
> Transmission Control Protocol, Src Port: 443, Dst Port: 54456, Seq: 0, Ack: 1, Len: 0

Frame 2 — SYN-ACK, source MAC = шлюз

Frame 3 — фінальний ACK (Seq=1, Ack=1, Len=0), з'єднання встановлене:



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.20.50.89	140.82.121.4	TCP	78	54456 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3736685868 TSecr=0 SACK_PERM
2	0.056079	140.82.121.4	10.20.50.89	TCP	74	443 → 54456 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1436 SACK_PERM TSval=2999834664 TSecr=373668...
3	0.056222	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=1 Ack=1 Win=132480 Len=0 TSval=3736685925 TSecr=2999834664
4	0.056368	10.20.50.89	140.82.121.4	TLSPV1...	386	Client Hello (SNI=github.com)
5	0.116946	140.82.121.4	10.20.50.89	TLSPV1...	1490	[TCP Previous segment not captured] , Ignored Unknown Record
6	0.116950	140.82.121.4	10.20.50.89	TLSPV1...	316	Ignored Unknown Record
7	0.116952	140.82.121.4	10.20.50.89	TCP	1490	[TCP Retransmission] 443 → 54456 [ACK] Seq=1 Ack=321 Win=67584 Len=1424 TSval=2999834722 TSecr=3736685925
8	0.117163	10.20.50.89	140.82.121.4	TCP	78	[TCP Dup ACK 3#1] 54456 → 443 [ACK] Seq=321 Ack=1 Win=132480 Len=0 TSval=3736685986 TSecr=2999834664 SLE...
9	0.117185	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=321 Ack=3099 Win=129408 Len=0 TSval=3736685986 TSecr=2999834722
10	0.119482	10.20.50.89	140.82.121.4	TLSPV1...	72	Change Cipher Spec
11	0.119498	10.20.50.89	140.82.121.4	TLSPV1...	124	Application Data
12	0.119550	10.20.50.89	140.82.121.4	TLSPV1...	152	Application Data
13	0.119584	10.20.50.89	140.82.121.4	TLSPV1...	124	Application Data
14	0.176914	140.82.121.4	10.20.50.89	TCP	66	443 → 54456 [ACK] Seq=3099 Ack=385 Win=67584 Len=0 TSval=2999834782 TSecr=3736685988
15	0.176916	140.82.121.4	10.20.50.89	TLSPV1...	145	Application Data
16	0.176918	140.82.121.4	10.20.50.89	TLSPV1...	145	Application Data
17	0.176919	140.82.121.4	10.20.50.89	TCP	66	443 → 54456 [ACK] Seq=3257 Ack=529 Win=67584 Len=0 TSval=2999834783 TSecr=3736685988
18	0.176921	140.82.121.4	10.20.50.89	TLSPV1...	130	Application Data
19	0.177092	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=529 Ack=3257 Win=130944 Len=0 TSval=3736686045 TSecr=2999834782
20	0.177136	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=529 Ack=3321 Win=130880 Len=0 TSval=3736686046 TSecr=2999834783
21	0.177383	10.20.50.89	140.82.121.4	TLSPV1...	97	Application Data
22	0.182653	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
23	0.182656	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
24	0.182658	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
25	0.182659	140.82.121.4	10.20.50.89	TLSPV1...	848	Application Data
26	0.182661	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data
27	0.182662	140.82.121.4	10.20.50.89	TLSPV1...	1490	Application Data

> Frame 3: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface en0
> Ethernet II, Src: 46:c0:a4:4b:9b:6b (46:c0:a4:4b:9b:6b), Dst: Google_43:f3:c8 (b0:e4:d5:43:f3:c8)
> Internet Protocol Version 4, Src: 10.20.50.89, Dst: 140.82.121.4
> Transmission Control Protocol, Src Port: 54456, Dst Port: 443, Seq: 1, Ack: 1, Len: 0

Frame 3 — ACK

Кінець тієї ж сесії — обмін [FIN, ACK] з обох боків і фінальний [RST] (закриття з'єднання):

Capturing from Wi-Fi: en0 (tcp and net 140.82.112.0/20)

Apply a display filter ... <#>

No.	Time	Source	Destination	Protocol	Length	Info
470	0.476677	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
471	0.476679	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
472	0.476681	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
473	0.476683	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
474	0.476684	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
475	0.476686	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
476	0.476687	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
477	0.476688	140.82.121.4	10.20.50.89	TLsv1...	1490	Application Data
478	0.476689	140.82.121.4	10.20.50.89	TLsv1...	415	Application Data
479	0.477172	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [ACK] Seq=560 Ack=595412 Win=905728 Len=0 TSval=3736686346 TSecr=2999835083
480	0.477301	10.20.50.89	140.82.121.4	TLsv1...	90	Application Data
481	0.477838	10.20.50.89	140.82.121.4	TCP	66	54456 → 443 [FIN, ACK] Seq=584 Ack=595412 Win=905728 Len=0 TSval=3736686346 TSecr=2999835083
482	0.535392	140.82.121.4	10.20.50.89	TLsv1...	90	Application Data
483	0.535395	140.82.121.4	10.20.50.89	TCP	66	443 → 54456 [FIN, ACK] Seq=595436 Ack=585 Win=67584 Len=0 TSval=2999835143 TSecr=3736686346
484	0.535691	10.20.50.89	140.82.121.4	TCP	54	54456 → 443 [RST] Seq=585 Win=0 Len=0
485	10.986257	140.82.114.22	10.20.50.89	TLsv1...	105	Application Data
486	10.986518	10.20.50.89	140.82.114.22	TCP	66	54024 → 443 [ACK] Seq=1 Ack=40 Win=2048 Len=0 TSval=3384666090 TSecr=3176316126
487	10.987037	10.20.50.89	140.82.114.22	TCP	66	54024 → 443 [FIN, ACK] Seq=1 Ack=40 Win=2048 Len=0 TSval=3384666090 TSecr=3176316126
488	11.104463	140.82.114.22	10.20.50.89	TLsv1...	90	Application Data
489	11.104466	140.82.114.22	10.20.50.89	TCP	66	443 → 54024 [FIN, ACK] Seq=64 Ack=2 Win=93 Len=0 TSval=3176316243 TSecr=3384666090
490	11.104699	10.20.50.89	140.82.114.22	TCP	54	54024 → 443 [RST] Seq=2 Win=0 Len=0
491	20.241090	10.20.50.89	140.82.113.25	TCP	54	52595 → 443 [ACK] Seq=1 Ack=1 Win=2048 Len=0
492	20.358088	140.82.113.25	10.20.50.89	TCP	66	[TCP ACKed unseen segment] 443 → 52595 [ACK] Seq=1 Ack=2 Win=76 Len=0 TSval=1139334012 TSecr=3789025396
493	35.122686	140.82.113.25	10.20.50.89	TLsv1...	91	Application Data
494	35.122837	10.20.50.89	140.82.113.25	TCP	66	[TCP Previous segment not captured] 52595 → 443 [ACK] Seq=2 Ack=26 Win=2048 Len=0 TSval=3789085397 TSecr...
495	35.123165	10.20.50.89	140.82.113.25	TLsv1...	95	Application Data
496	35.241512	140.82.113.25	10.20.50.89	TCP	66	443 → 52595 [ACK] Seq=26 Ack=31 Win=76 Len=0 TSval=1139348895 TSecr=3789085397

> Frame 1: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0

> Ethernet II, Src: 46:c0:a4:4b:9b:6b (46:c0:a4:4b:9b:6b), Dst: Google_43:f3:c8 (b0:e4:d5:43:f3:c8)

> Internet Protocol Version 4, Src: 10.20.50.89, Dst: 140.82.121.4

> Transmission Control Protocol, Src Port: 54456, Dst Port: 443, Seq: 0, Len: 0

0000

b0 e4 d5 43 f3 c8 46 c0 a4 4b 9b 6b 08 00 45 00

...C..F...K..E..

0010

00 40 00 00 40 00 00 06 f8 f4 0a 14 32 59 8c 52

..@..@...2Y.R

0020

79 04 d4 b8 01 bb 37 52 9b 20 00 00 00 00 b0 c2

y.....7R.....

0030

ff ff 1f ac 00 00 02 04 05 b4 01 03 03 06 01 01

.....

0040

08 0a de b9 4d 2c 00 00 00 00 04 02 00 00

...M,.....

Завершення сесії — FIN/ACK і RST

Звіт (підсумок)

Пункт	Результат
Сайт	github.com → 140.82.121.3/4
Хопів	10 (ICMP traceroute)
Шлюз	10.20.50.1 (en0)
MAC шлюзу	b0:e4:d5:43:f3:c8
Handshake	SYN → SYN-ACK → ACK (tcpdump, розділ 4; Wireshark, розділ 9)